Private Video Streaming with PIR

Design, implementation and evaluation of a video player whose server cannot tell what you watch

Project report

CS670, Indian Institute of Technology Kanpur
Instructor: Prof. Adithya Vadapalli
Group 9
Code: https://github.com/kartik3777/CS670

Member	Roll number	Role
Kartik	228070503	PIR core: XOR and LWE schemes, real-catalog benchmark
Sabhyasachi	240905	Player and streaming client, playback evaluation
Lokesh Kumar Moond	230606	Network layer and servers
Krishna Chandu Pujari	220832	Evaluation and security

November 2026

Contents

Abstract 2

1. Introduction and motivation 3

2. Background and threat model 3

3. System architecture 4

4. The two PIR schemes in detail 5

5. Streaming layer: the player 6

6. Network layer 7

7. Evaluation 8

8. Security analysis 10

9. Limitations 13

10. Future work 13

11. Conclusion 13

Appendix A: How to reproduce 14

Appendix B: Test status 14

Appendix C: References 14

Abstract

We built and measured a video streaming prototype in which the server cannot tell which video, or which part of a video, a viewer is fetching. The video catalog is cut into equal-size chunks, and the player fetches each chunk with Private Information Retrieval (PIR). We implemented two PIR schemes: a two-server XOR scheme that is private even against an unlimited attacker as long as the servers do not collude, and a single-server scheme based on Learning With Errors (LWE).

On a real 35 MB catalog of 97 chunks of 367 KB each, XOR PIR downloads 2 times the chunk size and LWE PIR moves about 8 times the chunk size (1.5 MB up and 1.5 MB down for one chunk). XOR keeps up with a 2-second chunk on links down to about 3 Mbit/s. LWE needs about 13 Mbit/s, and at 10 Mbit/s a chunk takes 2.56 s to fetch against 2 s of video.

PIR alone is not enough. An attacker who sees only the number and size of requests identifies the watched video 30% of the time in a 200-video catalog, against 10% for a popularity-only guess. A seek is detected in 98.7% of sessions. Padding the last batch with dummy chunks cuts the attack to 6.9% for 2% extra bandwidth, and constant-rate traffic cuts it to 0.7% for 8.75 times the bandwidth.

Main caveats. Network time is calculated from measured byte counts, not measured on a real network. The LWE code is our own and uses a non-cryptographic random generator. The traffic defences are simulated and not yet part of the player. The prototype has no TLS or authentication.

1. Introduction and motivation

What you watch reveals who you are. A list of videos can show health worries, political views, religion or sexual orientation. Encryption protects the bytes on the wire, but the streaming server still sees which video every viewer asks for. This project asks whether a server can serve video while learning nothing about which video that is.

Private Information Retrieval (PIR) is the cryptographic tool for this. A client fetches record number *i* from a database held by the server, and the server cannot tell which *i* was requested. The obvious solution, downloading the whole catalog, is far too expensive. Real PIR sends far less, but it costs extra bandwidth and server computation. Video is a hard case because playback has a deadline: each 2-second chunk must arrive before the previous one finishes playing.

Goals

1. Build working PIR for video chunks, with both a two-server and a single-server scheme.
2. Build a player that can buffer, batch, seek and verify chunks on top of PIR.
3. Measure whether playback keeps up on realistic links and what each scheme costs.
4. Test what PIR does not hide: the traffic pattern around the private requests.

Contributions

- A shared PIR package (`pir_core`) with a split client/server API used by the player, the network layer and the evaluation code.
- A second, independent PIR implementation with C kernels for server-time scaling up to 1 GiB catalogs.
- A seekable player with batching, stale-request discarding and SHA-256 verification.
- TCP servers (C++ baseline, Python XOR and LWE) with a loopback test.
- A threat model, attackers and countermeasures showing that the traffic pattern still leaks, and what it costs to fix it.
- A real-catalog measurement of both schemes on 97 chunks of 367 KB.

2. Background and threat model

PIR guarantees one thing: the server's view of a single request is independent of the index requested. It says nothing about how many requests are made, when, or how large they are.

2.1 Two-server XOR PIR

The database is copied to two servers that do not collude. To get record *i*, the client picks a random bit-vector q0 with one bit per record. It sets q1 equal to q0 with bit *i* flipped, and sends q0 to server 0 and q1 to server 1. Each server XORs together all records whose bit is 1 and returns the result. The client XORs the two answers. Every record except *i* appears in both sums and cancels, leaving record *i*.

Each vector alone is uniformly random, so a single server learns nothing, even with unlimited computing power. If the two servers share their vectors, q0 XOR q1 is a unit vector that points at *i*.

2.2 Single-server LWE PIR

With only one server, privacy must rest on a hard problem. We follow the SimplePIR design. The database is a matrix of bytes. A public matrix A is derived from a seed. The server computes a one-time hint H = DB·A, which the client downloads once. To fetch a column, the client sends A·s + e + Δ·u, where s is a fresh secret, e is small noise and u marks the wanted column. The server multiplies its database by this query. The client subtracts H·s and rounds to recover the data. Hiding the wanted column relies on the Learning With Errors problem being hard.

2.3 Threat model

Adversary	Capability	XOR	LWE
One honest-but-curious server	Sees every query with its time	Private (information-theoretic)	Private (computational, under LWE)
Colluding servers	Both servers pool their views	Broken (100% recovery)	Not applicable
Passive network observer	Sees packet sizes and timing	Same as one server	Same as one server
Malicious server	Returns wrong answers	Not protected	Not protected

What we want to protect: which video is watched, which chunk or timestamp is fetched (including seek targets), and, as a secondary goal, coarse session properties such as total length and seek behaviour. Out of scope: IP-level metadata, TLS and authentication, denial of service, and integrity against malicious servers.

3. System architecture

The system has four layers. The player asks for chunks by index, the PIR client turns each index into private queries, the servers answer without learning the index, and the player checks the result against a public manifest.

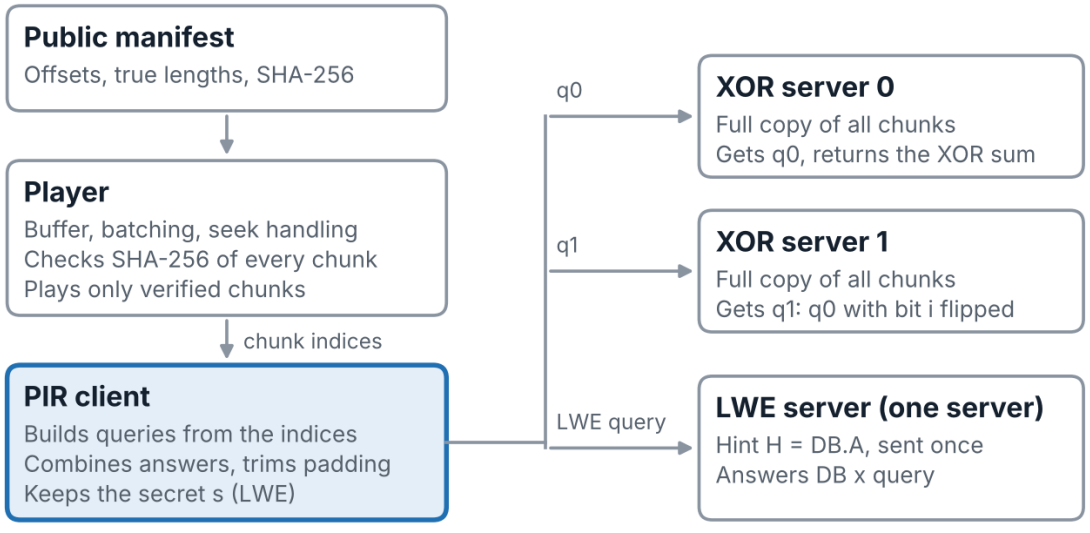


Figure 1. The PIR client sits between the player and the servers. For XOR it sends one query to each of two servers. For LWE it sends one query to one server.

3.1 Chunks, records and the manifest

- **Chunking.** Each video is cut into segments of about 2 seconds. In the real-catalog test one video became 97 chunks of 367 KB, 35 MB in total.
- **Equal-size records.** PIR needs every database entry to have the same size, otherwise the size of the answer would reveal the record. Short chunks are zero-padded to one catalog-wide size.
- **Manifest.** A public file lists, for every video, its offset in the global chunk index, the true length of each chunk and the SHA-256 hash of each padded record. Everyone downloads the same manifest, so it reveals nothing about one viewer.
- **Global index.** A chunk is identified by one number across the whole catalog. PIR hides that number.

3.2 Request path

1. The player decides it needs chunks *i1 … ik* (a batch of *k* chunks).
2. The PIR client builds the queries. For XOR it sends one bit-vector to each of two servers. For LWE it sends one numeric query to one server.
3. The servers process the whole database for every query. This full scan is what hides the index.
4. The client combines the answers, recovers the padded records and trims them to their true length.
5. The player compares the SHA-256 of each record with the manifest, and only then adds the chunk to its buffer.

3.3 Repository layout

Folder	Purpose
pir_core/	Shared PIR package: XOR and LWE, with a split client/server API
pir/	Independent PIR implementation with C kernels, the seekable player, playback evaluation
baseline/	Non-private HTTP chunk server and playback client for comparison
network/	C++ chunk server, Python TCP servers and client for XOR and LWE, loopback test
security_eval/	Threat model, session simulator, attackers, countermeasures, results
pir_core_extras/	Real-catalog benchmark script, tests for pir_core, raw run output
docs/	Literature survey, implementation plan and this report

4. The two PIR schemes in detail

Both schemes sit behind the same interface: `setup(records)` and `retrieve(indices)`, plus a split form with separate client and server objects so that the network layer can put a socket between them.

4.1 XOR scheme (two servers)

Item	Detail
Query (client to each server)	Bit-packed array of shape (k, ceil(N/8)) for a batch of k chunks over N records
Answer (server to client)	Array of shape (k, B), where B is the record size
Server work	XOR of all selected records, scanned in cache-sized blocks
Privacy	Each share alone is uniform; both shares together reveal the index
Data per chunk	0 KB up, 734 KB down for a 367 KB chunk (about 2 times)

Batching helps the server: it reads the database once and serves several queries from each pass. The independent implementation in `pir/` adds C kernels and measures about 70 ms of server time for one query over a 1 GiB catalog with 256 KiB records, which matches a scan speed of roughly 15 GB/s.

4.2 LWE scheme (one server)

Parameter	Value
Modulus q	2^32
Plaintext modulus p	2^8 (one byte per entry)
Scaling Δ	2^24
LWE dimension n	1024
Noise	Gaussian, standard deviation 6.4
Secret	Ternary, fresh for every query

The server stores the database as a matrix of bytes and derives the public matrix A from a seed. At setup it computes the hint H = DB·A. For the real 35 MB catalog the hint is about 23 MB, and the client downloads it once. A query is the vector A·s + e + Δ·u (one entry per database column); the answer is DB times the query. The client subtracts H·s and rounds to get the bytes back. The server does heavy work in exact float64 matrix products using 16-bit limbs, which lets it use fast BLAS routines without losing exactness.

One rule is critical: **a fresh secret for every query.** If a secret were reused, two queries could be subtracted to leak information. A test (`test_reusing_the_secret_would_leak`) demonstrates this.

4.3 Side by side

	XOR (2 servers)	LWE (1 server)
Trust assumption	Two servers do not collude	LWE is hard; no collusion needed
Privacy type	Information-theoretic	Computational
Up and down per 367 KB chunk	0 KB up, 734 KB down	1,504 KB up, 1,468 KB down
Total data vs chunk size	About 2 times	About 8 times
Server compute, batch of 1 to 16	10 to 134 ms	158 to 2,707 ms
One-time cost	None	Setup 1.6 s and hint download

4.4 Correctness testing

The shared package has 20 tests and the independent implementation has 16. They cover every index of a small database, batches with duplicate indices, batches that cross cache blocks, several database layouts for LWE, uniformity of XOR shares and LWE queries, decoding noise far below the threshold, rejection of bad input and of a wrong hint, and the secret-reuse leak. All retrieved chunks in the real-catalog run were compared byte for byte with the originals.

5. Streaming layer: the player

The player (`pir/player.py`) does not care how chunks arrive. It only needs a chunk source with `get_manifest()` and `fetch(video, index)`, plus a batch version. The same player runs on the plain HTTP baseline, the XOR backend and the LWE backend, so the only thing that changes between experiments is the retrieval step.

5.1 Buffering and batching

The player keeps a buffer of upcoming video and a small back buffer. When the buffer drops below a threshold, it fetches the next k chunks in one batch. Batching lets the server share work across queries. In the real-catalog run the cost per chunk is nearly flat in k for XOR, and for LWE it did not drop in a way we could separate from noise (about 158 ms per chunk at k=1 against about 170 ms at k=16).

5.2 Seeking

- **Inside the buffer.** The player jumps with no new request.
- **Short backward seek.** The back buffer serves it.
- **Far seek.** The player drops the buffer and fetches from the new position. Any request still in flight for the old position is tagged with an older generation number and discarded when it returns, so stale data never reaches the screen.

A cold seek costs about one startup delay plus the round that was thrown away.

5.3 Integrity check

Every chunk is checked with SHA-256 against the manifest before it is played. This catches decoding mistakes and corrupted answers. It does not add privacy, and since the manifest is not signed in the prototype it does not stop a malicious server that controls the manifest.

5.4 Non-private baseline

The `baseline/` folder holds a plain HTTP server and a playback client with the same metrics: startup latency, rebuffer count and time, fetch time and bytes. After one baseline session the server log contains the viewer's address, the video and the exact chunk sequence with timestamps. That log is what the PIR system must make uninformative.

6. Network layer

The network layer wraps the PIR core in TCP without changing either algorithm.

- **C++ baseline server.** Serves 1 MiB binary chunks over TCP with `PING` and `GET_CHUNK` commands. A response is a status byte, an 8-byte big-endian length and the payload, so the client can tell an error from a short transfer. The server validates the video id and chunk id and enforces a maximum size.
- **Python PIR servers.** XOR runs as two separate processes on two ports, each with an identical copy of the chunks. LWE runs as one process, which computes the hint at start-up.
- **Python client.** It sends the XOR query to each server separately, or the LWE query to its server, then decodes locally.
- **Chunks on disk.** `split_video.py` writes `chunk_0.bin`, `chunk_1.bin`, and so on; the server zero-pads short chunks to the largest size.

6.1 Loopback measurement

Fetching two 1 MiB chunks on one machine, four runs each, measured as client wall-clock time including process start-up.

Run	XOR (s)	LWE (s)
1	0.749	1.780
2	0.641	1.528
3	0.542	1.796
4	0.512	1.901
Mean	0.611	1.751

All downloaded chunks matched the originals byte for byte, and the padding was all zeros. These figures include process start-up and connection set-up, so they describe workflow time, not pure cryptographic cost.

6.2 Limits of the transport

The protocol has no TLS, authentication or replay protection and is meant for a trusted lab network. For two-server privacy, the two XOR servers must be run by independent parties, and the two query shares must never go to the same address. The LWE client downloads the hint on every run; caching it is an easy improvement. The network layer is not yet connected to the player: the player uses the PIR backends in-process, and a TCP-backed chunk source is the next step.

7. Evaluation

XOR PIR keeps up with playback down to about 3 Mbit/s, and LWE PIR needs about 13 Mbit/s. This section gives the evidence from three sources: the real-catalog run, the player experiments and the evaluation simulator. Read the caveats in section 9 before quoting any number.

7.1 Real catalog (97 chunks of 367 KB, 35 MB)

Compute is measured on one machine. Transfer time is calculated from the byte counts and the stated link speed, not measured on a real network. Each setting was run twice and the numbers moved by up to about 50%, so treat them as orders of magnitude.

Batch size (chunks)	XOR compute (ms)	LWE compute (ms)
1	10	158
4	37	480
8	73	1,100
16	123	2,707

Time to fetch one chunk, which must be below the 2 seconds of video it contains:

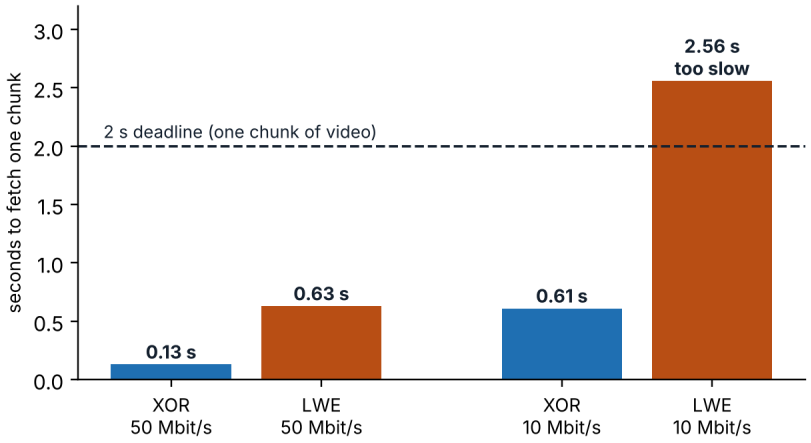


Figure 2. Measured compute plus calculated transfer, from the real-catalog run. The 10 Mbit/s LWE bar is above the deadline, so playback would stall.

From these numbers the break-even link speed, where fetch time equals playback time, is about 3 Mbit/s for XOR and about 13 Mbit/s for LWE.

7.2 Player experiments (pir/)

A 37.5 MiB catalog at about 1.05 Mbit/s with 128 KiB chunks and 40 ms latency, using the independent implementation and an emulated link. Mean startup time and rebuffering at batch size 1.

Link	XOR startup / rebuffers	LWE startup (first visit) / rebuffers
20 Mbit/s	0.30 s / 0	0.93 s (14.4 s) / 0
10 Mbit/s	0.51 s / 0	1.58 s (28.5 s) / 0
5 Mbit/s	0.93 s / 0	2.94 s (56.7 s) / 12
3 Mbit/s	1.49 s / 0	4.71 s (94.2 s) / 14

The first-visit figure includes the one-time hint download, which dominates LWE start-up. A bigger batch roughly doubles startup because the first batch must arrive before playback begins. Near-forward seeks inside the buffer cost nothing for XOR; for LWE they often miss the buffer because of the larger query.

7.3 Simulated five-minute sessions (security_eval/)

The simulator replays a 150-chunk (5 minute) video with 2-second chunks, 20 ms round-trip time and a fixed batch of 4, averaged over 12 sessions per cell. Its per-batch costs come from the real-catalog run.

Link	Plain: startup / rebuffer	XOR: startup / rebuffer	LWE: startup / rebuffer
5 Mbit/s	2.4 s / 0 s	5.1 s / 0 s	21.7 s / 490 s
10 Mbit/s	1.3 s / 0 s	2.9 s / 0 s	10.0 s / 74 s
25 Mbit/s	0.5 s / 0 s	1.0 s / 0 s	4.5 s / 0 s
50 Mbit/s	0.3 s / 0 s	0.5 s / 0 s	2.5 s / 0 s
100 Mbit/s	0.1 s / 0 s	0.3 s / 0 s	1.5 s / 0 s

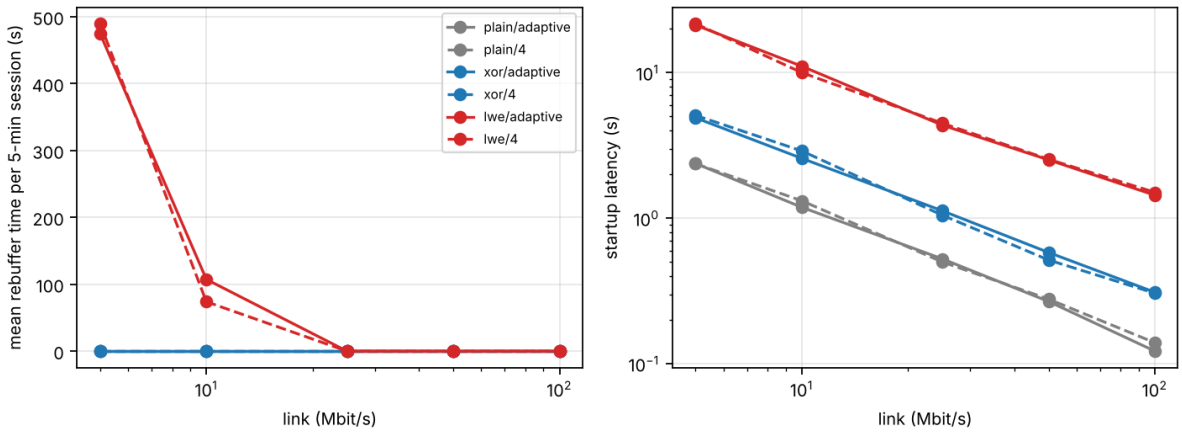


Figure 3. Simulated rebuffer time (left) and startup latency (right) against link speed. Solid lines: adaptive batch size; dashed: fixed batch of 4.

Data per session: plain about 52 MB, XOR about 105 MB (2.0 times), LWE about 215 MB down plus 220 MB up (about 8.3 times). Adaptive batching does not rescue LWE on slow links: it trades a few long stalls for many short ones.

7.4 Server cost against catalog size

With 32 KB chunks and a batch of 4 on a 2-core machine, XOR server time is about 1 ms for a 3 MB catalog and 11 ms for 64 MB, growing roughly linearly. LWE is 12 ms at 3 MB and 57 ms at 32 MB. The independent implementation measured XOR at about 70 ms per query for a 1 GiB catalog. Larger catalogs than these were not run; any larger figure is a straight-line estimate.

For the 35 MB catalog and a batch of 8, XOR needs about 72 ms of compute per 16 seconds of video (0.45% of a core) and LWE about 960 ms (6% of a core). Compute alone would let one core serve roughly 220 XOR viewers or 17 LWE viewers of this small catalog. Real capacity is lower once I/O and memory are counted.

8. Security analysis

The PIR schemes behave as designed, but the traffic pattern still gives the video away. We tested both questions with attackers that we implemented (`security_eval/eval/attacks.py`).

8.1 Is the PIR itself private?

Test	Result	Chance or expected
XOR, one server guesses a set bit (97 chunks, 20,000 trials)	1.06%	1.03% (±0.14%)
XOR, one server guesses an unset bit	1.01%	1.03%
XOR, one server, 1,000 chunks	0.13%	0.10% (±0.044%)
XOR, both shares combined (2,000 trials)	100%	1.03%
LWE, chi-square of the query at the wanted column (3,000 queries)	287	255 ± 23
LWE, chi-square at an unrelated column	262	255 ± 23
LWE, attacker picks the largest entry	1.4%	1.67%

The XOR results match the theory: one share is uniformly random, so the best guess is 1 in 97. With both shares the answer is exact, which is why the servers must be independent. The LWE test is a sanity check only. It used a reduced dimension (n=128) and a small database, and a statistical test cannot prove LWE security.

8.2 Does the traffic pattern leak?

A server or network observer sees, for each request, its time and its batch size. Together these give the number of requests R and the total chunks requested T. The total follows the length of the video, and a short last batch shows where the video ended. Our attacker knows the catalog lengths and popularity, learns what traffic each video produces by simulating the client, and picks the most likely video.

Catalog size	Attacker (XOR)	Attacker (LWE)	Popularity-only guess	Uniform chance
10 videos	45.7%	54.0%	28.0%	10%
50 videos	39.1%	36.6%	15.3%	2%
200 videos	29.7%	25.6%	10.0%	0.5%
1,000 videos	11.8%	9.9%	6.5%	0.1%

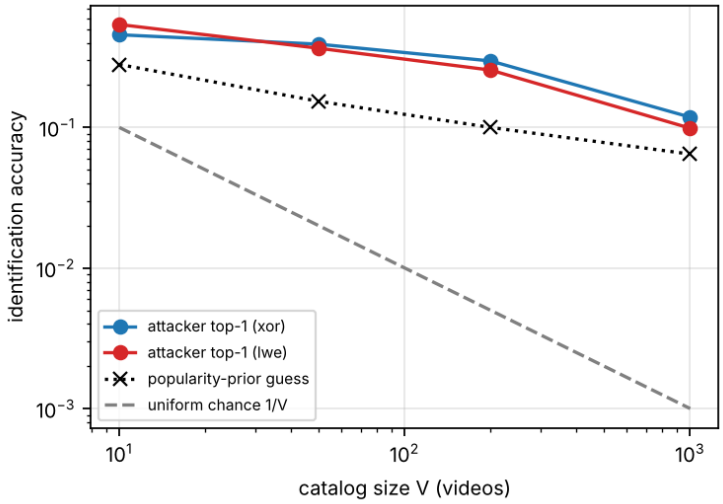


Figure 4. Video identification accuracy from the traffic alone, against catalog size (log scales). The leak is the same for XOR and LWE.

The leak is the same for both schemes because it sits in the traffic, not in the PIR. Accuracy falls as the catalog grows, but stays well above the popularity guess.

Seeks. When the viewer seeks, the player flushes its buffer and sends the next request earlier than the steady schedule of k × 2 seconds. The seek detector flags this gap. It catches 98.7% of seeks with 0% false alarms (area under the ROC curve 1.00). A constant-rate client is at 0.50, which is chance.

8.3 Countermeasures and what they cost

200 videos, XOR backend. Overhead is chunks sent divided by chunks needed.

Client configuration	Attacker success	Bandwidth overhead
None	27.5%	1.00 times
Pad the last batch	6.9%	1.02 times
Pad and round length to 25 chunks	3.2%	1.16 times
Round to 50 chunks	2.0%	1.30 times
Round to 100 chunks	1.7%	1.65 times
Round to 400 chunks	0.8%	4.98 times
Constant rate, no rounding	9.6%	1.71 times
Constant rate, round to 100 slots	0.7%	8.75 times

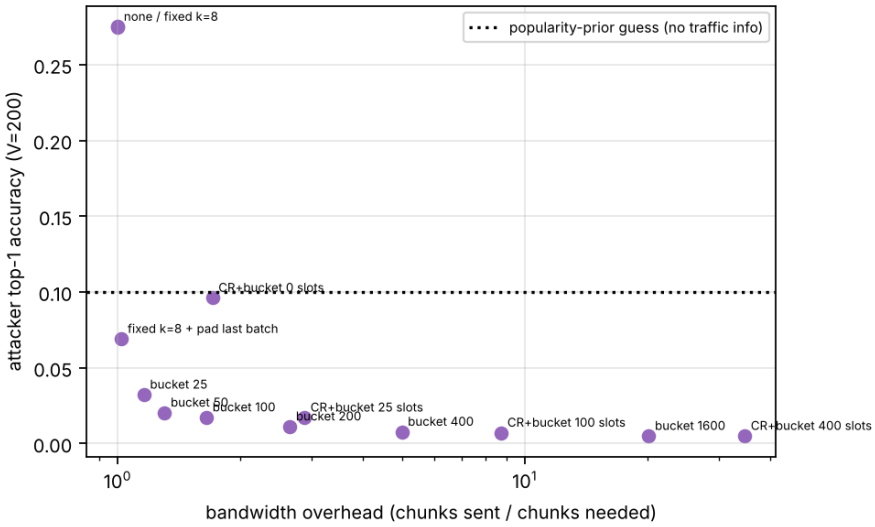


Figure 5. Attacker success against bandwidth overhead for each client configuration (200 videos, log x axis). The dotted line is the popularity-only guess.

Padding the last batch is almost free and takes the attacker below the 10% popularity guess, so it should always be on. Rounding the length trades more bandwidth for more privacy. Constant-rate traffic is the only option that also hides seeks, at about 1.9 times the bandwidth for seek hiding alone and 2.5 to 5 times the seek latency. On XOR it is affordable on every tested link. On LWE it doubles an already large bill and is unusable below about 25 Mbit/s.

Adaptive batch sizing helps playback on bad links but its batch sequence reveals the viewer's network speed, so padded modes should use a fixed batch size.

8.4 Comparison with earlier systems

	Popcorn (NSDI 2016)	Addra (OSDI 2021)	Ours, XOR	Ours, LWE
Workload	About 8,000 titles, 10,000 clients	32,000 users, 96-byte messages	1 viewer, 97 chunks of 367 KB	Same
Reported cost	1080 times compute, 14 times I/O	682 times ciphertext expansion	2.0 times bandwidth	About 8 times bandwidth

These are not like-for-like. The earlier systems report large multi-user deployments; ours is a single-viewer prototype on one machine. The defensible reading is about bandwidth only.

9. Limitations

- **Network time is modelled.** Transfer time is calculated from byte counts and a stated link speed. No real two-machine network was used, and the two XOR servers ran on one machine in the source measurements.
- **Small catalog.** The real run used one 35 MB video. Catalogs above 64 MB were not run for LWE, and larger figures are straight-line estimates.
- **Noisy timings.** LWE compute varied by up to about 50% between two runs. The numbers show orders of magnitude, not precise values.
- **Different set-ups.** The player experiments used a 1 Mbit/s stream with 128 KiB chunks. The simulator used 2-second chunks of 367 KB. Compare trends, not absolute values, across them.
- **LWE is not hardened.** The code uses the NumPy PCG64 generator for the secret and noise. A real system needs a cryptographic generator and a check of the parameters with a lattice estimator.
- **No integrity or transport security.** A malicious server can corrupt answers, there is no TLS or authentication, and the manifest is unsigned.
- **Attacker model.** The simulated attacker uses only the request count and total chunks. Real attackers could use finer timing, link several sessions or use outside knowledge. The viewing model (Zipf popularity, 50% full views) is an assumption.
- **Single bitrate.** A catalog with several bitrates would add another distinguishing feature.

10. Future work

1. Connect the TCP servers to the player with a network-backed chunk source, then test on a real network with two independent XOR hosts.
2. Run larger catalogs, with sharding on the server side.
3. Replace the random generator, check LWE parameters with a lattice estimator, and cache the hint on the client.
4. Add integrity: a signed manifest, answer verification and TLS.
5. Build last-batch padding into the real player, since it costs about 2%.
6. Test stronger attackers that use timing and cross-session linking.

11. Conclusion

Private video streaming with PIR works at small scale. Two-server XOR PIR costs about twice the data, plays smoothly on links down to about 3 Mbit/s, and needs two servers that do not work together. Single-server LWE PIR removes that trust requirement but moves about eight times the data and needs roughly 13 Mbit/s or more. PIR hides what is requested, not how much is requested: the traffic pattern identified the video 30% of the time in a 200-video catalog. Padding the last batch removes most of that leak for about 2% extra bandwidth, and stronger defences are available at a higher cost.

Appendix A: How to reproduce

The code is at https://github.com/kartik3777/CS670. Needs Python 3 with numpy, pandas and matplotlib. A C compiler is optional; the C kernels fall back to numpy.

```
./run_all_checks.sh                         # all test suites + network loopback, 1-2 min
export PYTHONPATH=$PWD
cd baseline && python3 run_demo.py          # non-private baseline
cd ../pir && python3 eval_playback.py       # playback grid
cd ../security_eval && python3 -m eval.run_all   # attackers + countermeasures, ~5 min
cd .. && BW_MBPS=10 python3 pir_core_extras/run_catalog.py xor seg/video1   # real catalog
```

Network servers: see `network/README_NETWORK.md`.

Appendix B: Test status

Suite	Tests
pir_core (shared package)	20
pir/test_pir.py (independent PIR)	16
pir/test_player.py (seekable player)	15
pir/test_pircore_adapter.py (pir_core drives the player)	1
security_eval/eval/test_eval.py	6
Network loopback (XOR two servers + LWE)	1 check

Appendix C: References

- Chor, Goldreich, Kushilevitz, Sudan. Private Information Retrieval. Journal of the ACM, 1998.
- Henzinger, Hong, Corrigan-Gibbs, Meiklejohn, Vaikuntanathan. One Server for the Price of Two: Simple and Fast Single-Server Private Information Retrieval (SimplePIR). USENIX Security 2023.
- Gupta, Panda, Popa, Katz, Ristenpart. Popcorn: Bringing Private Video Streaming to Netflix-scale Rates. NSDI 2016.
- Ahmad, Kim, Mohassel, Rindal. Addra: Metadata-private voice communication over fully untrusted infrastructure. OSDI 2021.
- Project repository: https://github.com/kartik3777/CS670.
- Project files: security_eval/Final_Report_Evaluation_and_Security.md, security_eval/threat_model.md, pir_core_extras/results/real_catalog_run.txt.